

SUAVE thruster monitoring and adaptation

Shown for the BehaviorTree manager (suave_bt). The first page is the full sequence. The following pages show each of the four phases separately, with notes on every step. Step numbers match across all diagrams.

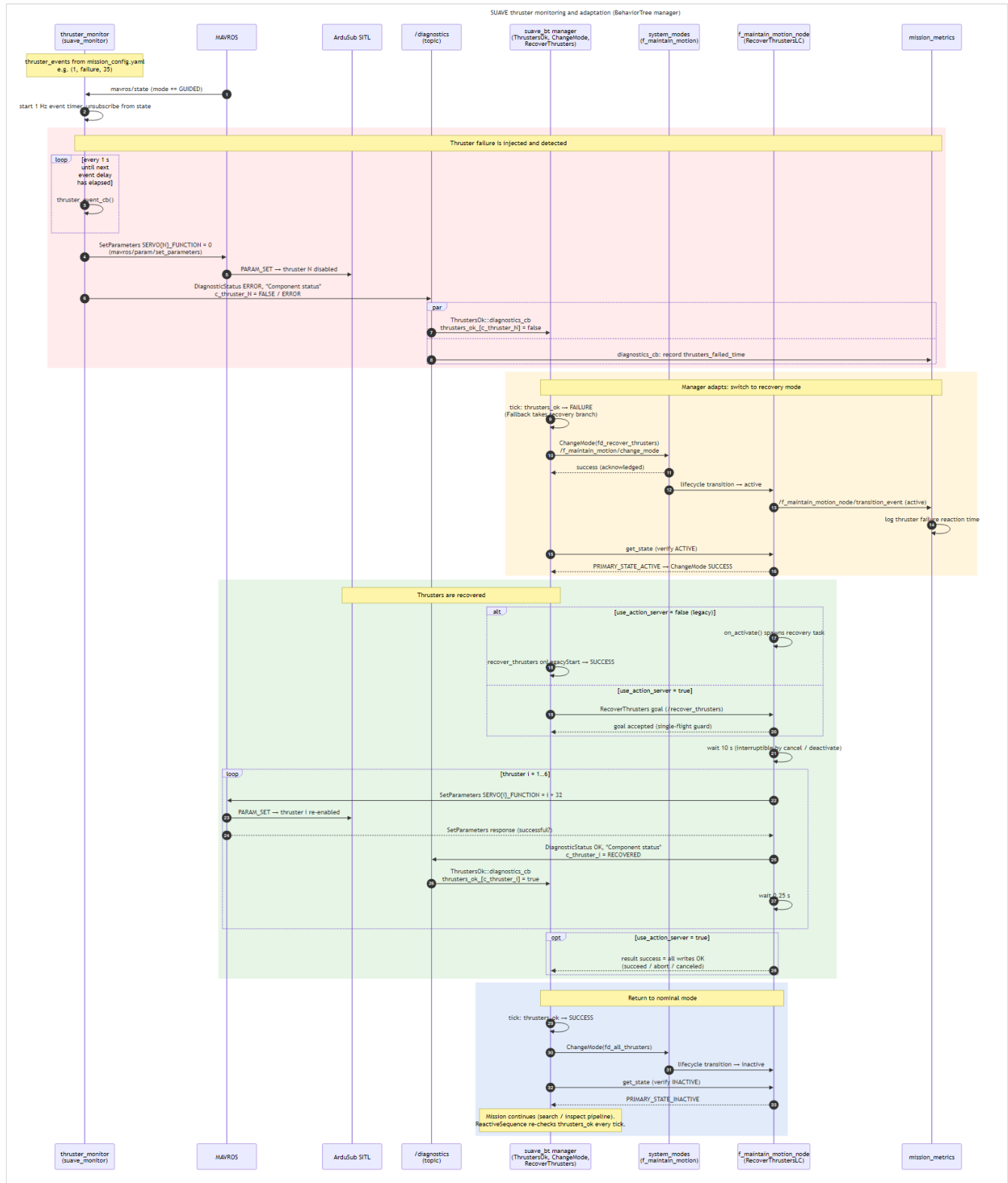


Figure 1. Full sequence. A thruster failure is injected and reported on `/diagnostics` (phase 1). The manager switches `f_maintain_motion` to its recovery mode (phase 2). The recovery node re-enables the thrusters through MAVROS (phase 3). The manager then returns to the normal mode (phase 4).

Phase 1: thruster failure is injected and detected (steps 1–8)

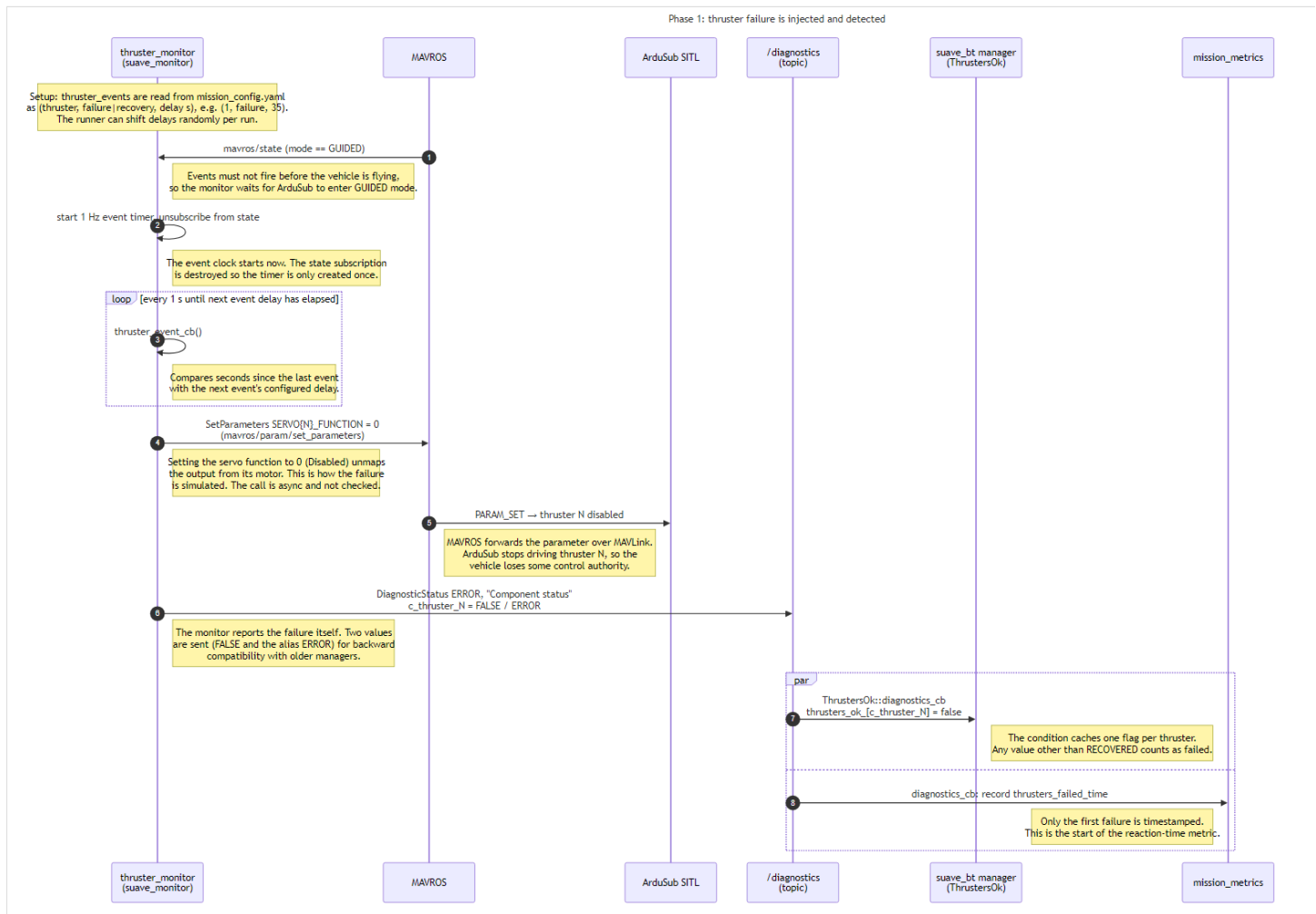


Figure 2. `thruster_monitor` injects a failure on a schedule and reports it.

- The monitor waits until MAVROS reports `GUIDED` mode, so no failure happens before the vehicle is under way. It then checks the next `thruster_events` entry from `mission_config.yaml` once a second.
- When that event is due, the monitor sets `SERVO{N}_FUNCTION = 0` through MAVROS. ArduSub then stops driving that thruster. Nothing breaks in Gazebo; the failure is only a parameter change.
- The monitor publishes `c_thruster_N = FALSE` (and the alias `ERROR`) on `/diagnostics`.
- The BT's `ThrustersOk` condition marks that thruster as failed. `mission_metrics` records the time of the failure, which starts the reaction-time measurement.

Phase 2: manager adapts by switching to the recovery mode (steps 9–16)

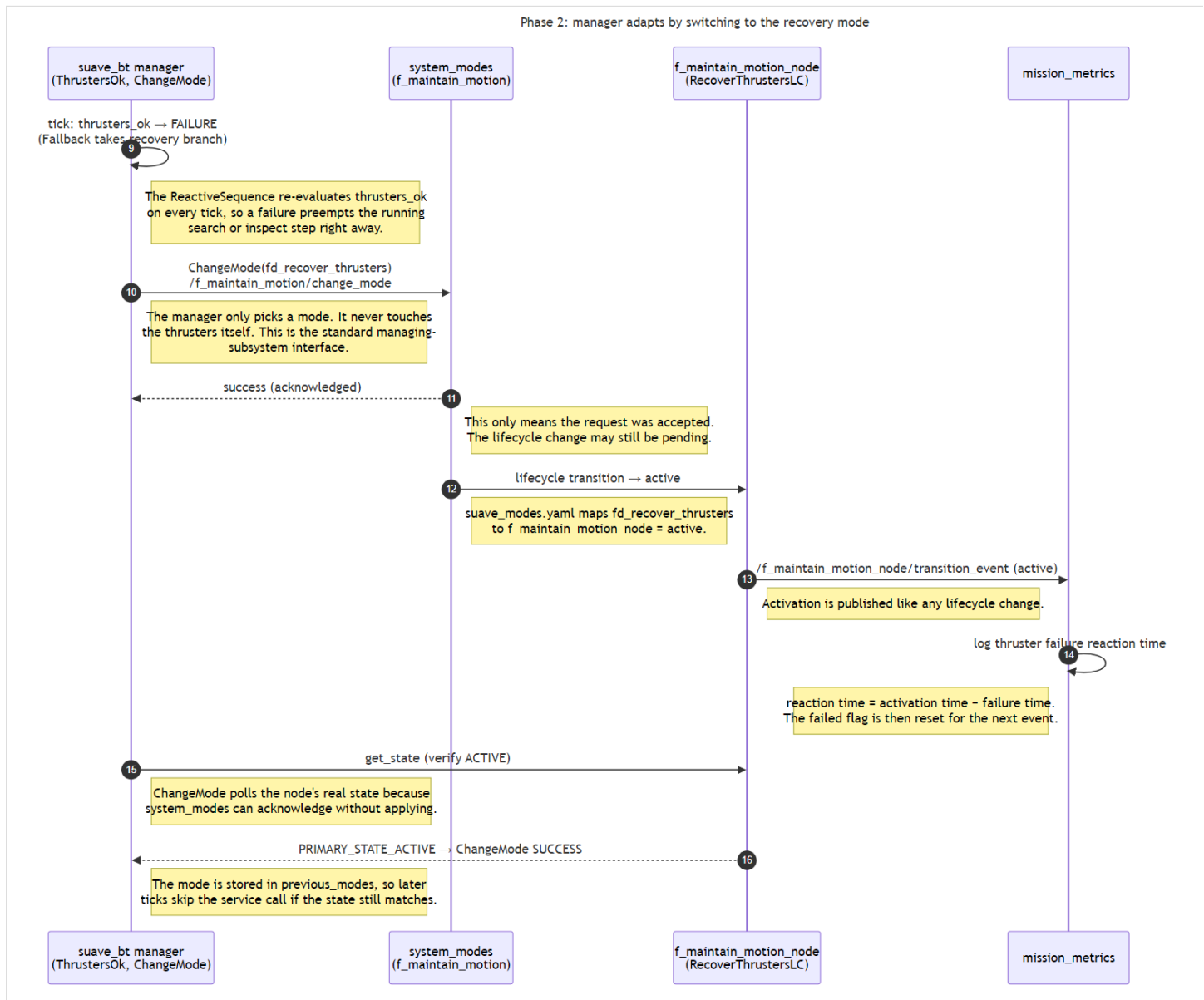


Figure 3. The manager reacts by changing modes; it never touches the thrusters itself.

- `thrusters_ok` is re-checked on every tick, so the failure immediately preempts the current search or inspection. The Fallback takes the recovery branch.
- The manager calls `/f_maintain_motion/change_mode` with `fd_recover_thrusters`. According to `suave_modes.yaml`, this mode makes `system_modes` activate `f_maintain_motion_node`.
- `system_modes` can accept a request before it has actually applied it. `ChangeMode` therefore queries the node with `get_state`, and succeeds only once the node is `ACTIVE`.
- The activation shows up on the node's `transition_event`. `mission_metrics` logs the reaction time: activation time minus failure time.

Phase 3: thrusters are recovered (steps 17–28)

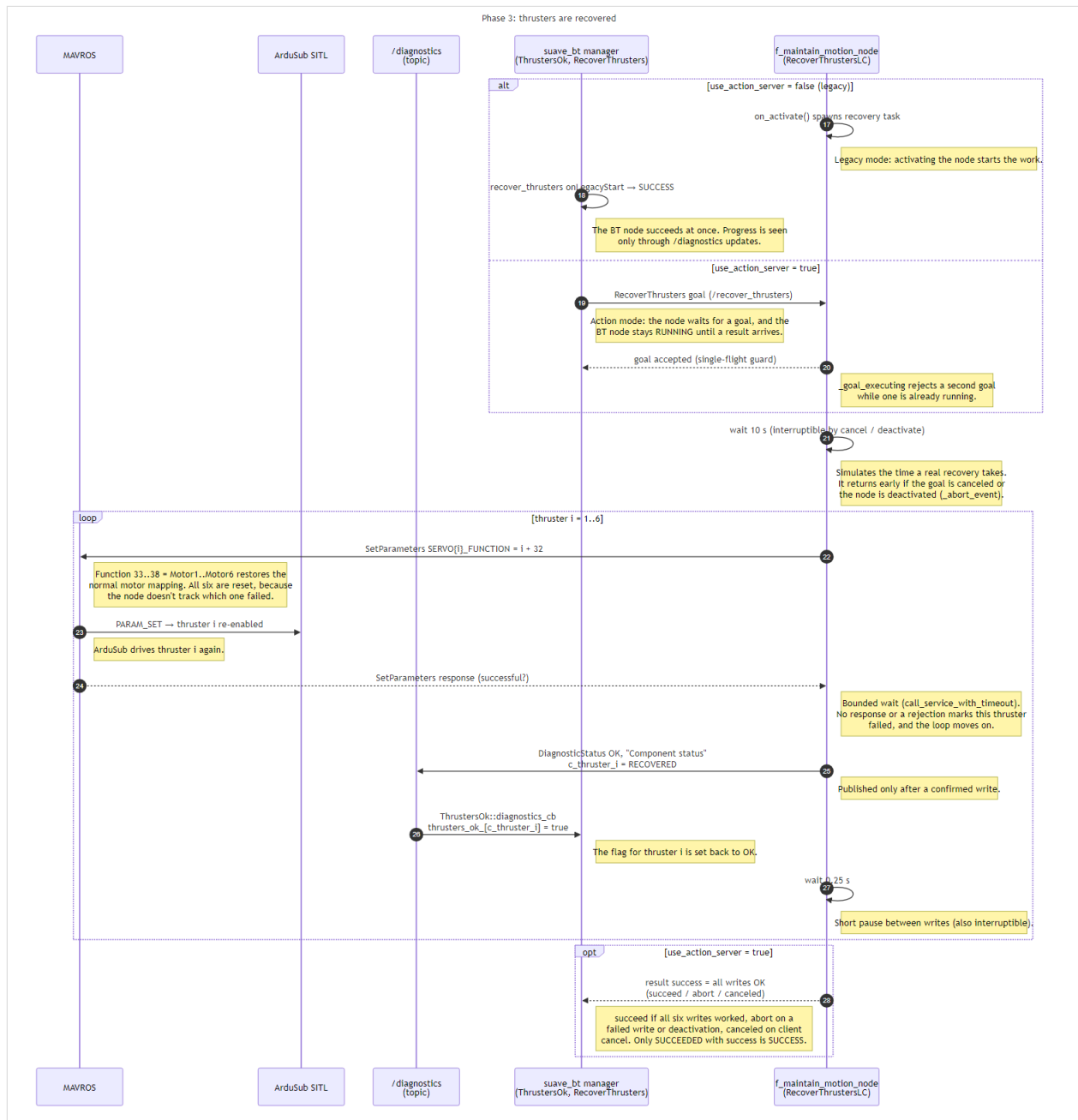


Figure 4. RecoverThrustersLC re-enables all six thrusters through MAVROS.

- **How recovery starts.** By default (`use_action_server = false`), activating the node starts recovery, and the BT's `recover_thrusters` step succeeds at once. In action mode, the BT sends a `RecoverThrusters` goal and waits for the result.
- The node waits 10 s to stand in for real repair time. It then sets `SERVO{i}_FUNCTION = i + 32` (Motor1–Motor6) for every thruster, because it doesn't track which one failed.
- After each write that MAVROS confirms, it publishes `c_thruster_i = RECOVERED`, and `ThrustersOK` marks that thruster as OK again.
- All waits stop early if the goal is cancelled or the node is deactivated. In action mode the result is `succeed` if all writes worked, `abort` on a failed write or deactivation, and `canceled` on a client cancel.

Phase 4: return to the normal mode (steps 29–33)

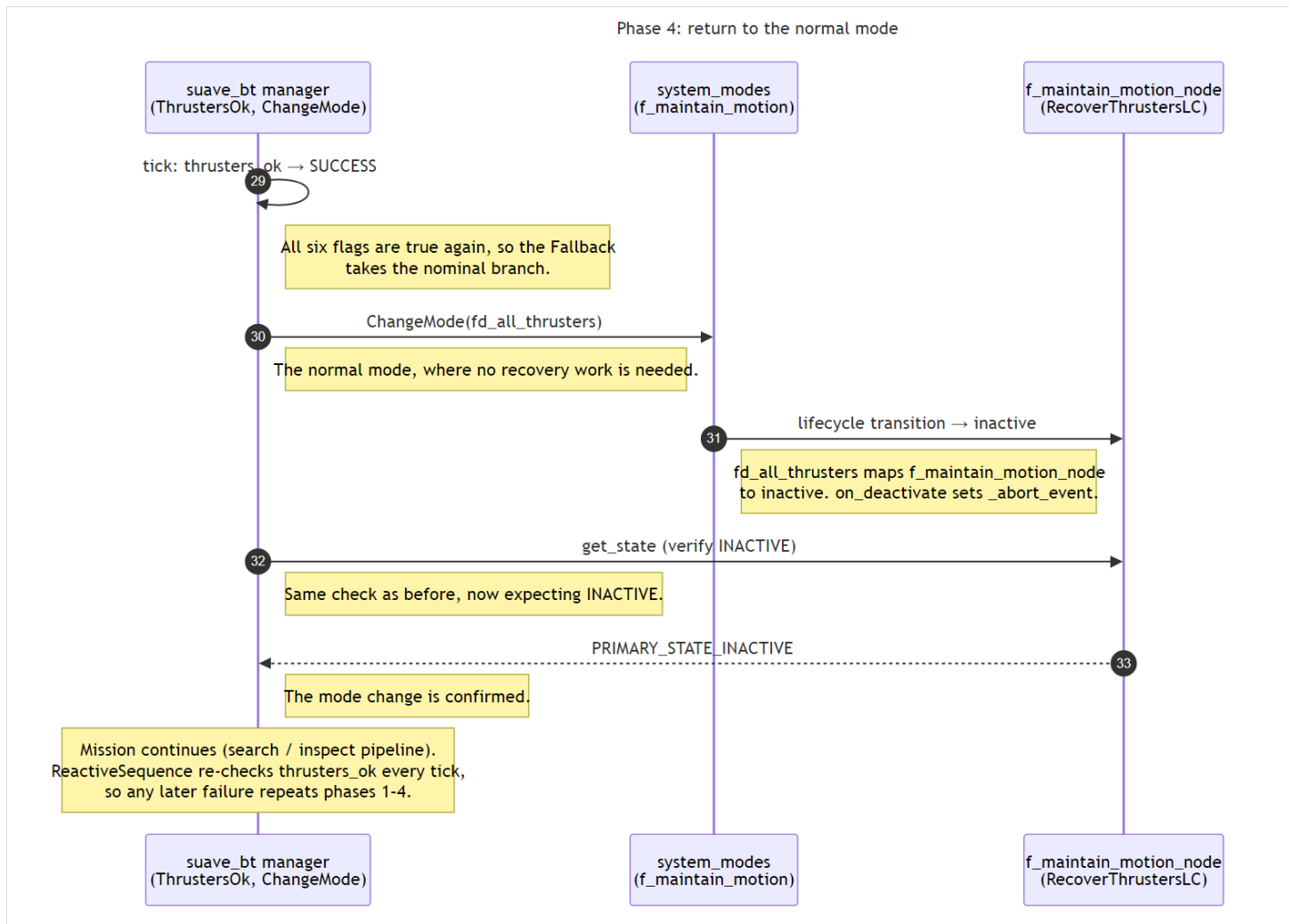


Figure 5. The manager returns to the normal mode once all thrusters report OK.

- All six thruster flags are OK again, so `thrusters_ok` succeeds and the Fallback takes the normal branch.
- The manager requests `fd_all_thrusters`, which makes `system_modes` deactivate `f_maintain_motion_node`. Deactivation also sets `_abort_event`, which stops any recovery still running.
- As in phase 2, `ChangeMode` calls `get_state` to confirm the node is actually `INACTIVE`.
- The mission continues with searching and inspecting the pipeline. `thrusters_ok` is still checked on every tick, so a later failure runs through phases 1–4 again.